

Sistema de Recuperación Semántica sobre el Corpus CODEFEST AD ASTRA 2026

The Data Alchemists – Universidad Icesi

Informe técnico - Etapa 1: Construcción de la Base de Conocimiento

En la primera etapa de este reto nos enfocaremos en evaluar qué tan eficazmente el sistema identifica los documentos y fragmentos relevantes dentro del corpus a partir de una pregunta formulada en lenguaje natural. De esta definición se desprende el principio que orienta las decisiones descritas en las siguientes páginas: en ninguna etapa del proceso, ya sea durante la indexación o la búsqueda, interviene un modelo generativo. El funcionamiento del sistema se sustenta exclusivamente en modelos encoder y en operaciones numéricas realizadas sobre representaciones vectoriales.

Este documento presenta y justifica las cuatro decisiones principales que establece la guía: la estrategia de chunking, la selección del modelo encoder, el tipo de índice utilizado en FAISS y la incorporación de un grafo de conocimiento. Asimismo, se proporciona el contexto necesario para comprender cada una de estas decisiones, así como las excepciones y ajustes que surgieron durante su implementación.

Flujo del sistema, de un vistazo

Documentos ADL	→	Extracción y limpieza	→	Chunking (fragmentos)	→	Encoder (embeddings)	→	Índice FAISS	→	Búsqueda y agregación
----------------	---	-----------------------	---	-----------------------	---	----------------------	---	--------------	---	-----------------------

Los primeros cinco pasos construyen la base vectorial una sola vez; el último, búsqueda y agregación, se repite cada vez que llega una pregunta nueva, sin volver a tocar nada de lo anterior.

1. El corpus y su preparación

El corpus fue proporcionado por la organización. Está compuesto por 1,826 archivos, correspondientes a los tres fenómenos del reto y distribuidos entre aproximadamente 21 observatorios, cada uno con diferentes formas de publicar y estructurar la información. En la práctica, esto implicó trabajar con siete formatos de archivo distintos dentro de las mismas carpetas, lo que requirió definir mecanismos específicos de extracción de texto para cada formato.

Formato	Cantidad	Método de extracción
PDF	759	pypdf, preservando el orden de lectura de párrafos; descriptado automático cuando el archivo lo requería; se eliminaron cabeceras y pies de página repetidos
JSON	954	Identificación del esquema de cada archivo y extracción de los campos narrativos, dejando los campos descriptivos (url, fecha, autores) como metadata
CSV	26	Lectura de la fila de cabecera y recorrido fila a fila, conservando cada valor junto al nombre de su columna
Excel (XLSX)	4	Datasets del observatorio AI_Index_Stanford; se procesaron como pares columna:valor por fila (recomendación de la guía) y se sumaron al índice ya construido
Imagen	8	OCR con Tesseract: 4 con texto útil se incorporaron al corpus, 4 sin texto legible quedaron documentadas sin inventar contenido
Texto plano	1	Extracción directa
PBF (mapas)	74	Excluidos desde el diseño: son coordenadas y capas geográficas, no contenido narrativo que un buscador semántico pueda aprovechar

De los 1,826 archivos que conformaban el corpus inicial, 1,681 documentos quedaron listos para su procesamiento en la primera pasada. Los 155 archivos restantes quedaron fuera de esta etapa, cada uno por un criterio verificable y documentado en exclusiones.csv: 73 correspondían a los PBF ya mencionados; 48 eran PDF escaneados sin capa de texto; 17 eran manifiestos internos del scraper utilizado para construir el corpus (listas de URLs y hashes, no artículos); 8 eran imágenes y 4 archivos de Excel que requerían un tratamiento específico posterior; 2 PDF presentaban un flujo de datos corrupto, debido a un error de lectura del stream del archivo; 1 archivo estaba vacío; 1 CSV correspondía también a un manifiesto sin contenido indexable; y 1 archivo de imagen en formato .avif no fue reconocido por el pipeline de extracción, diseñado para los formatos estándar del corpus. En todos los casos, la exclusión de la primera pasada respondió a un criterio técnico verificable, ausencia de texto legible, de contenido indexable o de soporte para el formato, y no a una decisión de conveniencia.

Las 8 imágenes se sometieron posteriormente a un proceso adicional de verificación, que establece la aplicación de OCR a este tipo de archivos antes de descartarlos. Para ello, se utilizó Tesseract para la extracción de texto. Cuatro imágenes arrojaron texto legible, con una extensión que iba desde un título de portada de 7 palabras hasta una tabla de 269 palabras, por lo que se incorporaron al corpus como documentos nuevos y se sumaron al índice ya construido, sin modificar los fragmentos existentes. Las cuatro imágenes restantes correspondían a fotografías sin texto legible y se mantuvieron excluidas, con el motivo documentado en cada caso.

Los 4 archivos de Excel del observatorio AI_Index_Stanford recibieron igualmente un tratamiento específico. Se procesaron siguiendo la recomendación de la guía para hojas de cálculo, mediante una representación de cabecera más pares columna:valor por fila, y se incorporaron al índice como 721 fragmentos adicionales.

A cada documento se le asignó un identificador único (doc_id), construido a partir del índice de archivos proporcionado por la organización, con el fin de garantizar la trazabilidad y evitar la asignación arbitraria de identificadores. La validación de este proceso reveló un caso que requería un tratamiento explícito: 59 nombres de archivo se repetían en distintas subcarpetas de un mismo observatorio, aunque correspondían a contenidos diferentes. Por ejemplo, un mismo nombre de archivo de CSET aparecía tanto en la carpeta Reports como en Translation, con dos identificadores oficiales diferentes.

Para resolver esta ambigüedad, se cruzaron de forma simultánea la carpeta y el nombre exacto del archivo, garantizando que cada documento conservara su identificador oficial sin asociaciones incorrectas. Finalmente, el campo fuente, utilizado para la comparación con el ground truth, conserva en todos los casos el nombre original del archivo tal como fue entregado por la organización, independientemente del doc_id asignado durante el procesamiento.

2. Estrategia de chunking y su justificación

La guía plantea cinco estrategias posibles para la fragmentación del texto: tamaño fijo de tokens, oración, párrafo, estructura jerárquica del documento y fragmentación semántica con superposición entre fragmentos consecutivos. La estrategia de tamaño fijo se descartó por diseño, ya que establece los límites a partir de un contador de palabras sin considerar la integridad de las ideas; además, la guía exige que ningún fragmento contenga una oración incompleta. La fragmentación por párrafos o por estructura del documento tampoco resultaba viable de manera consistente. Tras la extracción de texto desde formatos como PDF, JSON y CSV, la noción de “párrafo” varía considerablemente entre documentos, por ejemplo, una fila de CSV no constituye un párrafo, lo que habría generado fragmentos con tamaños muy dispares.

Se adoptó una combinación de fragmentación por oración con una ventana de superposición de una oración entre fragmentos consecutivos. Cada fragmento agrupa oraciones consecutivas hasta aproximarse a un objetivo de 250 palabras, sin dividir ninguna oración. La última oración de cada fragmento se repite como primera oración del siguiente, con el propósito de preservar el contexto en los límites entre fragmentos. El objetivo de 250 palabras se estableció de manera deliberada, ya que coincide con el límite máximo exigido por la guía para cada fragmento del archivo de resultados. Definir este límite desde la etapa de *chunking* permite evitar recortes posteriores que pudieran alterar la integridad de los fragmentos.

La validación del *chunking* sobre el corpus completo permitió identificar un caso límite que requería una regla adicional: bloques superiores a 2,000 palabras en los que el detector de fin de oración no lograba realizar una segmentación adecuada. Estos casos correspondían principalmente a listados extensos de nombres de empresas separados por comas, en los que abreviaturas como “Inc.” eran interpretadas como finales de oración. Para resolverlos, se estableció un límite máximo de 375 palabras por oración, acompañado de una regla de segmentación en cascada: primero por comas y, cuando esto no fuera suficiente, por palabras. Esta regla garantiza el cumplimiento del límite incluso en los casos en que la detección convencional de oraciones falla.

Con estas reglas, el procesamiento de los 1,681 documentos produjo 125,961 fragmentos. El fragmento más corto contiene 1 palabra y el más largo 425 palabras, mientras que tanto el promedio como la mediana se sitúan en 230 palabras. Ningún fragmento supera las 600 palabras.

Finalmente, la guía exige ocho campos por fragmento almacenado. Los ocho campos se encuentran presentes y verificados en *metadata.jsonl*. Como parte de esta validación, se corrigió el tipo de dato del campo *fenomeno*, pasando de texto a entero en la totalidad de las líneas, sin modificar el índice FAISS ni ninguno de los demás campos.

3. Selección del encoder y criterios de elección

La guía exige justificar el encoder frente a seis criterios concretos: soporte multilingüe, dimensionalidad del vector, longitud máxima de entrada, rendimiento en benchmarks públicos, licencia y eficiencia computacional.

El modelo seleccionado fue *intfloat/multilingual-e5-base*, publicado en Hugging Face bajo licencia MIT, con 278 millones de parámetros. Frente a los seis criterios establecidos por la guía:

Criterio de la guía	Cómo lo cumple multilingual-e5-base
Soporte multilingüe	Entrenado sobre decenas de idiomas, entre ellos español, inglés y portugués, justo los tres del corpus
Dimensionalidad del vector	768 dimensiones, un tamaño intermedio que no dispara el costo de almacenamiento ni de búsqueda para aproximadamente 126.000 vectores
Longitud máxima de entrada	512 tokens; el objetivo de 250 palabras por fragmento se pensó para quedar cómodo bajo ese límite
Rendimiento en benchmarks	La familia E5 figura entre los modelos multilingües con mejor desempeño documentado en recuperación densa (MTEB/BEIR) en su rango de tamaño
Licencia	MIT, una de las licencias que la guía prefiere explícitamente
Eficiencia computacional	278M de parámetros corren en CPU en tiempos razonables, el equipo no cuenta con GPU dedicada

Un aspecto técnico central para el uso correcto de E5 es que el modelo distingue entre el texto que se busca y el texto sobre el que se realiza la búsqueda. Por ello, requiere un prefijo específico para cada caso: “passage:” para los fragmentos del corpus y “query: ” para las preguntas. La aplicación consistente de estos prefijos en ambas direcciones fue parte integral de la implementación, ya que omitirlos o invertirlos puede degradar la calidad de la búsqueda, aun cuando el modelo utilizado sea técnicamente el mismo.

La guía permite además combinar varios encoders y fusionar sus resultados. Esta alternativa fue evaluada, pero se optó por utilizar un único encoder multilingüe. Al cubrir los tres idiomas del corpus, incorporar un segundo modelo habría duplicado el tiempo de generación de embeddings, cerca de una hora sobre 125,961 fragmentos, sin una ganancia clara que lo justificara. Además, habría introducido una capa adicional de fusión de resultados y, con ella, una superficie de error evitable. Por tanto, se trató de una decisión de alcance deliberada, respaldada por el análisis costo-beneficio del tiempo disponible, y no de una limitación técnica.

4. Índice FAISS y módulo de recuperación

Se construyó un índice FAISS de tipo IndexFlatIP sobre vectores normalizados, equivalente en la práctica a ordenar por similitud coseno. La guía sugiere este tipo de índice como suficiente para el volumen de documentos del reto y reserva los índices aproximados, como IVFFlat o HNSW, para corpus de mayor escala o situaciones en las que el tiempo de respuesta sea crítico. En este caso, la búsqueda sobre más de 126,000 vectores de 768 dimensiones mediante un índice plano toma milisegundos, por lo que no resultaba necesario sacrificar exactitud a cambio de una velocidad que el sistema no requería.

El índice y sus metadatos quedaron almacenados en `base_vectorial/encoder_multilingual-e5-base/`, manteniendo una correspondencia exacta entre el orden de `metadata.jsonl` y los identificadores internos asignados por FAISS a cada vector. Tras incorporar los fragmentos recuperados mediante OCR y, posteriormente, los correspondientes a los 4 archivos de Excel, el índice quedó conformado por 126,687 vectores y la metadata por 126,687 líneas, conservando esta correspondencia en todo momento.

El módulo de recuperación (`recuperacion.py`) codifica la pregunta utilizando el mismo encoder empleado para construir el índice y el prefijo “query: ”. Posteriormente, realiza la búsqueda en FAISS y construye los 3 documentos de salida agrupando los primeros 60 candidatos por `doc_id` y asignando a cada documento el puntaje de su fragmento con mayor similitud.

Ningún modelo generativo interviene en este flujo. El orden de los resultados depende exclusivamente de la similitud coseno calculada por FAISS, mientras que el texto de cada fragmento se entrega exactamente como aparece en el documento original, sin resumirlo ni reescribirlo. Cuando un fragmento candidato supera las 250 palabras, se divide en subfragmentos que respetan el corte por oración completa y conservan el mismo `chunk_id`. Esta lógica se validó sobre 596 fragmentos reales antes de aplicarse al corpus completo, confirmando que ningún subfragmento supera el límite establecido.

5. Grafo de conocimiento (componente bonus)

Además del buscador por significado, se construyó el componente opcional de grafo de conocimiento: una red de entidades, personas, organizaciones y lugares, conectadas mediante relaciones extraídas del propio corpus. Mientras que la búsqueda vectorial identifica fragmentos similares en significado a la pregunta, el grafo permite recuperar fragmentos conectados mediante menciones explícitas. Ambos mecanismos se complementan dentro del mismo flujo de recuperación.

El grafo se construye en tres etapas. Primero, un modelo de reconocimiento de entidades (NER) recorre cada fragmento e identifica personas, organizaciones y lugares mencionados. Se seleccionó Davlan/distilbert-base-multilingual-cased-ner-hrl, publicado en Hugging Face bajo licencia AFL-3.0, con cobertura de español, inglés y portugués, entre otros diez idiomas, y un tamaño liviano que permite un procesamiento más rápido que el encoder de recuperación. La licencia fue un criterio explícito de selección: el Q&A oficial del reto establece que licencias como CC BY-NC-SA 4.0 no están permitidas para ningún componente del proyecto, incluido el bonus, por lo que este criterio se consideró desde la evaluación inicial de las alternativas de NER. Segundo, cuando dos entidades aparecen juntas dentro del mismo fragmento, se registra una relación entre ellas, cuyo peso corresponde al número de fragmentos distintos en los que ambas coinciden. Tercero, el resultado se almacena como un grafo mediante NetworkX y se exporta a grafo/grafog.graphml, el formato exigido por la guía.

Por diseño, no se construyó un segundo modelo para clasificar el tipo de relación entre dos entidades, por ejemplo, distinguir entre “desarrolla” y “regula”, ya que esta tarea requeriría entrenar o adaptar un modelo específico de extracción de relaciones, con una complejidad adicional de validación. En su lugar, se adoptó la co-ocurrencia dentro del mismo fragmento como relación genérica (“se menciona junto a”), una heurística que la guía permite explícitamente. Aunque esta relación es menos granular que una relación tipada, representa únicamente aquello que el sistema puede verificar: que dos entidades comparten contexto, sin inferir una relación causal que no puede establecerse a partir del corpus.

El grafo no reemplaza la búsqueda vectorial, sino que la complementa. Cuando llega una pregunta, el mismo modelo NER identifica las entidades mencionadas y, si alguna existe en el grafo, se recuperan los fragmentos donde aparece directamente y los correspondientes a sus vecinos de primer orden. Esta lista se combina con la obtenida mediante FAISS utilizando Reciprocal Rank Fusion (RRF), con la constante de suavizado estándar ($k_0 = 60$). El diseño garantiza que el componente bonus no comprometa el sistema base: si la carpeta grafo/ no existe, generador.py opera exactamente igual que sin el grafo.

El grafo se construyó sobre el corpus real completo y posteriormente se extendió con los fragmentos de los archivos de Excel recuperados, sin reprocesar lo ya construido. En total, contiene 414,942 entidades y 11,043,918 relaciones, y cubre 126,682 de los 126,687 fragmentos del índice. Los 5 fragmentos recuperados mediante OCR quedan fuera de esta cobertura, una fracción marginal que se documenta por precisión.

La escala del grafo introdujo dos ajustes de ingeniería durante la fase de validación. Primero, algunas entidades de alta frecuencia, como nombres de países y organismos internacionales, generan miles de conexiones, por lo que la búsqueda de vecinos se limitó a los 30 de mayor peso por entidad, priorizando las conexiones con mayor señal. Segundo, el archivo grafog.graphml final tiene un tamaño de 2.34 GB, por lo que su lectura en formato de texto (XML) resultaba poco práctica en cada ejecución de generador.py. Para resolverlo, se incorporó una copia en formato binario (grafo/checkpoint.pkl, 627 MB) que el sistema utiliza automáticamente para cargar el grafo en segundos, mientras que grafog.graphml se conserva como archivo oficial de entrega en el formato exigido por la guía. Con estos ajustes, resultados.jsonl se generó utilizando el grafo y el índice en su versión final.

6. Reproducibilidad y limitaciones

La guía establece un criterio inapelable: si generador.py no reproduce resultados.jsonl, la entrega queda fuera de la evaluación. generador.py depende de recuperacion.py, que a su vez utiliza chunking.py para dividir correctamente los fragmentos largos y, cuando existe la carpeta grafo/, de grafog_recuperacion.py. Aunque el árbol de ejemplo de la guía menciona explícitamente solo generador.py, el funcionamiento del

sistema en un computador distinto al original requiere estos tres archivos adicionales. Por ello, los cuatro se incluyen en la carpeta de entrega.

El proceso completo es reproducible desde cero en cualquier computador con **Python 3** y librerías gratuitas y de código abierto: pypdf, cryptography, numpy, faiss-cpu, sentence-transformers, networkx, transformers, pytesseract y pillow. Estas dos últimas requieren además la instalación de **Tesseract** en el sistema, por ejemplo, mediante `brew install tesseract tesseract-lang` en Mac.

```
1. python3 extraer_corpus.py corpus salida_completa
2. python3 chunking.py salida_completa/documentos_extraidos.jsonl chunks.jsonl
3. python3 generar_embeddings.py chunks.jsonl embeddings_checkpoints
4. python3 construir_indice.py embeddings_checkpoints base_vectorial multilingual-e5-base
4b. python3 extraer_imagenes.py corpus salida_completa/exclusiones.csv salida_completa
    inventario_doc_id.csv
4c. python3 chunking.py salida_completa/nuevos_documentos_ocr.jsonl nuevos_chunks.jsonl
4d. python3 generar_embeddings.py nuevos_chunks.jsonl nuevos_embeddings_checkpoints
4e. python3 agregar_al_indice.py base_vectorial/encoder_multilingual-e5-base
    nuevos_embeddings_checkpoints
4f. (bonus) python3 construir_grafo.py chunks.jsonl grafo
4g. python3 extraer_excel_pendientes.py corpus salida_completa/exclusiones.csv
    salida_completa inventario_doc_id.csv
4h. python3 chunking.py salida_completa/nuevos_documentos_excel.jsonl
    nuevos_chunks_excel.jsonl
4i. python3 generar_embeddings.py nuevos_chunks_excel.jsonl
    nuevos_embeddings_excel_checkpoints
4j. python3 agregar_al_indice.py base_vectorial/encoder_multilingual-e5-base
    nuevos_embeddings_excel_checkpoints
4k. (bonus) python3 extender_grafo.py nuevos_chunks_excel.jsonl grafo
5. python3 generador.py
```

El paso 4f, de carácter opcional, ya se ejecutó sobre el corpus completo. Una reconstrucción completa en un equipo sin GPU dedicada requiere varias horas; sin embargo, el script guarda el progreso periódicamente y permite reanudar el proceso ante cualquier interrupción. Los pasos 4g–4k siguen el mismo patrón de extensión incremental para los 4 archivos de Excel recuperados: estos se procesaron, se incorporaron al índice con 721 fragmentos nuevos y se extendió el grafo ya construido con ellos, sin reprocesar la información existente.

El alcance de esta entrega se definió con criterios explícitos, y las siguientes fronteras forman parte de esa definición. De las 8 imágenes recuperadas, 4 no contenían texto legible incluso después de aplicar OCR, por lo que quedaron documentadas fuera del índice. Los 48 PDF escaneados sin capa de texto se excluyeron por el mismo criterio. Por su parte, el grafo de conocimiento cubre **126,682 de los 126,687** fragmentos del índice; los 5 fragmentos recuperados mediante OCR quedan fuera de esta cobertura, una fracción marginal que se documenta por precisión.

El sistema tampoco aplica deduplicación entre documentos con contenido solapado. Se revisaron las 50 respuestas y se identificó este patrón en 21 de ellas, siempre entre `doc_id` distintos y nunca como

duplicados del mismo chunk_id dentro de una misma respuesta. Esto confirma que el origen corresponde a un solapamiento genuino presente en el corpus fuente y no a un error de indexación.

Finalmente, el sistema se validó con las **50 preguntas oficiales**, sin contar con un conjunto de referencia propio que permitiera calcular **NDCG@10** o **F1@3** antes de la evaluación oficial de la organización.